SUMMER INTERNSHIP AT TOTAL PLANT ENGINEERING

TITLE: DEVELOPMENT OF AN ULTRASONIC ANEMOMETER

INTRODUCTION

The Ultrasonic Anemometer is a wind measurement instrument designed to determine wind speed and wind direction using the ultrasonic transit-time principle. The system employs four 40 kHz ultrasonic transducers arranged in two perpendicular axes, enabling accurate measurement without the use of moving mechanical parts. This design improves reliability, reduces maintenance, and provides a faster response compared to conventional mechanical anemometers.
This product is developed by modifying the DL1GLH Ultrasonic Anemometer Prototype II. The original controller has been replaced with an ESP32-S3 microcontroller, and the complete electronic schematic and PCB have been redesigned using KiCad. The hardware consists of a 15 V power supply section, ultrasonic transmitter and receiver circuits, signal conditioning stages, analog multiplexers, and communication interfaces.

PROBLEM STATEMENT

Conventional mechanical anemometers use rotating components to measure wind speed and direction, making them susceptible to friction, wear, and regular maintenance. These limitations can reduce measurement accuracy and reliability over time, particularly in harsh environmental conditions. The objective of this project is to develop an Ultrasonic Anemometer that measures wind speed and wind direction using ultrasonic transit-time measurement. The system is designed with an ESP32-S3 microcontroller and a custom PCB to provide a compact, reliable, and maintenance-free solution suitable for future fabrication, testing, and environmental monitoring applications.

PROPOSED SYSTEM

The proposed system is an Ultrasonic Anemometer designed to measure wind speed and wind direction using the ultrasonic transit-time measurement principle. The system employs four 40 kHz ultrasonic transducers, an ESP32-S3 microcontroller, transmitter and receiver circuits, signal conditioning circuits, and analog multiplexers to perform accurate wind measurements.

The hardware is powered by a 15 V DC input, which is regulated to provide the required voltage levels for both analog and digital circuits. The ESP32-S3 controls the transmission and reception of ultrasonic signals, processes the received data, and calculates the wind parameters. A custom two-layer PCB has been designed using KiCad, and the Gerber files have been generated for PCB fabrication.

COMPONENTS USED

COMPONENT	QUANTITY
ESP32-S3 DevKitC-1U	1
LM7812 Voltage Regulator	1
LM7805 Voltage Regulator	1
LM7905 Voltage Regulator	1
LD1117V33 Voltage Regulator	1
NME1212SC Isolated DC-DC Converter	1
ICL7667 Driver ICs	2
CD4052B Analog Multiplexers	2
LT1001 Precision Operational Amplifier	1
LM833 Operational Amplifiers	1
78601_1C Pulse Transformers	4
40 kHz Ultrasonic Transducers	4
DB9 Connector	1
Passive Components	As required

COMPONENTS DESCRIPTION

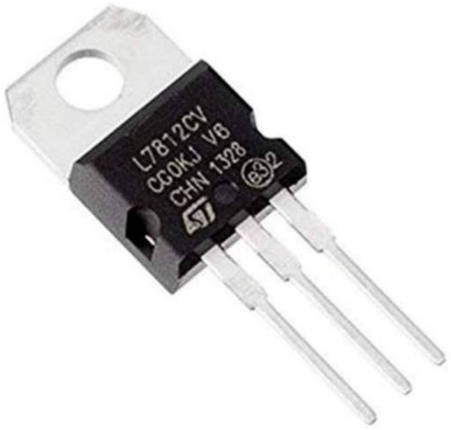
1. ESP32-S3 DevKitC-1U

The ESP32-S3 DevKitC-1U is the main controller of the Ultrasonic Anemometer. It controls the transmission and reception of ultrasonic signals, selects the transmitting and receiving channels through the analog multiplexers, processes the received signals, and performs the calculations required to determine wind speed and wind direction. The ESP32-S3 also manages serial communication with external devices through the DB9 interface.



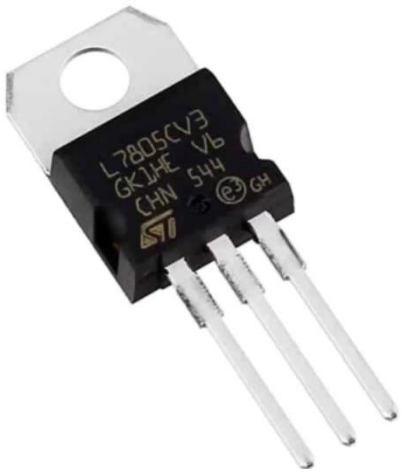
2. LM7812 Voltage Regulator

The LM7812 is a fixed positive voltage regulator used to convert the 15 V DC input into a stable +12 V supply. This regulated voltage powers the ultrasonic driver circuitry and other hardware modules requiring a 12 V supply.



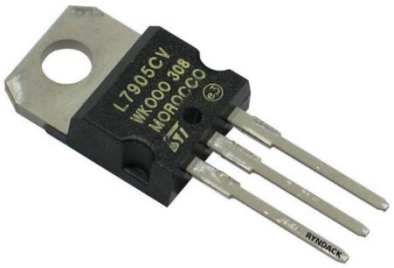
3. LM7805 Voltage Regulator

The LM7805 provides a regulated +5 V output from the +12 V supply. It powers the analog multiplexers, operational amplifiers, and other analog components that require a stable 5 V operating voltage.



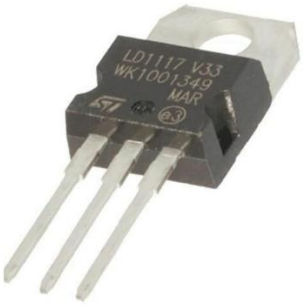
4. LM7905 Voltage Regulator

The LM7905 is a negative voltage regulator that generates a regulated −5 V supply. This negative voltage is required for the operational amplifier circuits to ensure proper amplification of bipolar ultrasonic signals.



5. LD1117V33 Voltage Regulator

The LD1117V33 is a low-dropout voltage regulator that converts the +5 V supply into a stable +3.3 V output. It provides the operating voltage required by the ESP32-S3 microcontroller and other digital circuits.



6. NME1212SC Isolated DC-DC Converter

The NME1212SC is an isolated DC-DC converter that provides electrical isolation between the power source and sensitive analog circuitry. The isolation helps reduce electrical noise and improves signal integrity during ultrasonic signal processing.



7. ICL7667 Driver IC

The ICL7667 is a high-speed dual MOSFET driver used to generate high-current drive pulses for the ultrasonic transmission circuit. It amplifies the control signals from the ESP32-S3 and drives the pulse transformers that excite the ultrasonic transducers.



8. CD4052B Analog Multiplexer

The CD4052B is a dual 4-channel analog multiplexer used to select the transmitting and receiving ultrasonic channels. The ESP32-S3 controls the multiplexer selection lines, allowing different transducer pairs to be connected during ultrasonic measurements.



9. OP07 Precision Operational Amplifier

The OP07 is a precision operational amplifier used in the receiver circuit to amplify the weak ultrasonic signals received by the transducers. It offers low offset voltage, low noise, and high stability, making it suitable for precision signal amplification.



10. LM833 Operational Amplifier

The LM833 is a low-noise dual operational amplifier used for additional signal amplification and filtering. It improves the quality of the received ultrasonic signal before it is processed by the ESP32-S3.



11. 78601_1C Pulse Transformer

The 78601_1C pulse transformer is used in the transmission circuit to provide pulse isolation and voltage transformation. It transfers the excitation pulses from the driver ICs to the ultrasonic transducers while maintaining electrical isolation and efficient signal transfer.



12. 40 kHz Ultrasonic Transducers

The system uses four 40 kHz ultrasonic transducers arranged in two perpendicular axes. These transducers alternately transmit and receive ultrasonic pulses. The variation in the ultrasonic transit time caused by airflow is used to calculate wind speed and wind direction.

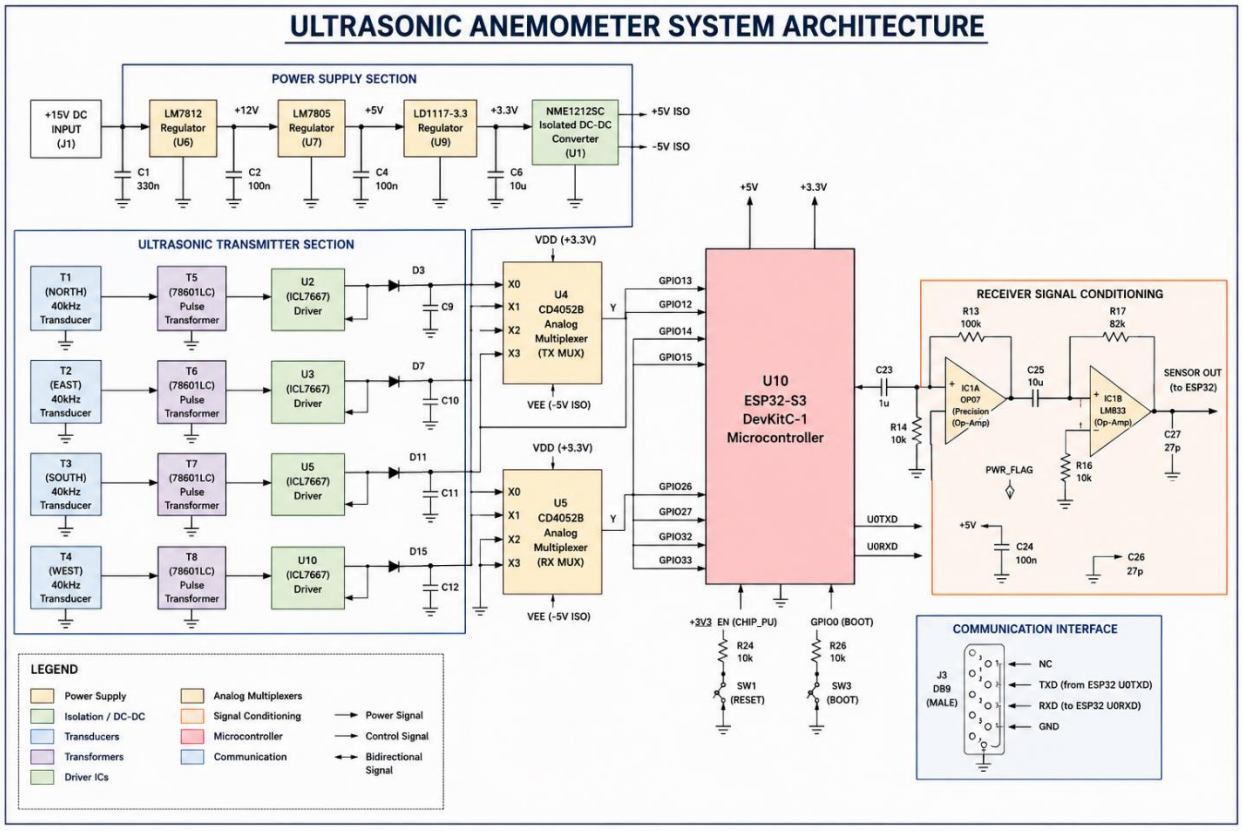


13. DB9 Connector

The DB9 connector provides a serial communication interface between the Ultrasonic Anemometer and an external computer or monitoring system. It enables the transfer of measured data and supports future integration with external software.



SYSTEM ARCHITECTURE



Connection

1. Microcontroller Unit (ESP32-S3-DevKitC-1)

The ESP32-S3 (U10) functions as the central processing unit of the ultrasonic anemometer. It controls ultrasonic pulse transmission, measures signal transit time, processes the received data, and communicates the calculated wind speed and wind direction to external systems.

• GPIO Control for Ultrasonic Transmission

- **GPIO13:** Connected to the address control input of the upper CD4052B analog multiplexer (U4) for selecting the transmitting ultrasonic channel.
- **GPIO12:** Connected to the address control input of the lower CD4052B analog multiplexer (U5) for selecting the receiving ultrasonic channel.
- **GPIO1 and GPIO2:** Used as multiplexer control lines for selecting one of the four ultrasonic transducers (TX1–TX4).

• UART Communication

- **U0TXD:** Connected to Pin 2 (TX) of the DB9 connector (J3).
- **U0RXD:** Connected to Pin 3 (RX) of the DB9 connector (J3).

The UART interface enables communication between the ESP32-S3 and an external monitoring or data acquisition system.

• Power Supply

- **VIN:** Receives regulated +5 V from the LM7805 voltage regulator.
- **3V3:** Powered through the LD1117-3.3 V regulator.
- **GND:** Connected to the common system ground.

• Reset Circuit

- **EN (CHIP_PU):** Connected to +3.3 V through a 10 kΩ pull-up resistor (R24) and to the reset pushbutton (SW1), allowing manual reset of the ESP32-S3.

- **GPIO0 (BOOT):** Connected through a 10 kΩ pull-up resistor (R26) and boot pushbutton (SW3) for firmware programming mode.

2. Ultrasonic Transmission Section

The transmission section generates high-voltage ultrasonic excitation pulses required for driving the 40 kHz ultrasonic transducers.

• Ultrasonic Driver ICs (ICL7667)

- **U2 and U3 (ICL7667):** Operate as high-current MOSFET drivers.
- Receive logic signals from the CD4052B analog multiplexer.
- Amplify the control signal to provide sufficient current for driving the pulse transformers.

• Pulse Transformers

- **T5, T6, T7 and T8 (78601LC):**
  - Primary winding is driven by the ICL7667 driver IC.
  - Secondary winding generates the high-voltage excitation pulse.
  - Electrically isolates the driver stage from the ultrasonic transducers.

• Protection Network

- Diodes **D3–D16** provide protection against transformer flyback voltage.
- Capacitors **C10–C13** provide AC coupling and pulse shaping.
- Resistors **R1–R8** limit current and improve pulse stability.

3. Ultrasonic Receiving Section

The receiving section detects weak ultrasonic signals arriving from the opposite transducer and amplifies them for digital processing.

• Analog Multiplexer (CD4052B)

- **U5:** Selects the active receiving transducer.
- Routes the received ultrasonic signal to the signal conditioning circuit.

• OP07 Precision Operational Amplifier

- Provides the first-stage amplification of the received ultrasonic signal.
- Offers low offset voltage and high stability for accurate signal amplification.

• LM833 Operational Amplifier

- Performs additional gain and filtering.
- Improves the signal-to-noise ratio before the signal reaches the ESP32-S3.

• Signal Conditioning Components

- Capacitors **C23–C31** provide AC coupling and frequency filtering.
- Resistors **R13–R22** determine amplifier gain and filter characteristics.

4. Analog Multiplexer Section (CD4052B)

Two CD4052B analog multiplexers are used to control the transmission and reception paths of the ultrasonic transducers.

• U4 – Transmission Multiplexer

- Selects one of the four transformer outputs.
- Routes the transmit pulse to the selected ultrasonic transducer.

• U5 – Reception Multiplexer

- Selects the receiving transducer.
- Routes the received ultrasonic signal to the amplifier circuit.

5. Power Supply Section

The power supply generates all regulated voltages required for the ultrasonic anemometer.

• LM7812 Voltage Regulator

- Converts the +15 V input supply into a regulated +12 V output.
- Powers the ICL7667 driver ICs and analog circuitry.

• LM7805 Voltage Regulator

- Converts +12 V into a regulated +5 V supply.
- Powers the ESP32-S3 DevKit, analog multiplexers, and operational amplifiers.

• LD1117-3.3 V Regulator

- Converts +5 V into +3.3 V.
- Supplies the ESP32-S3 logic circuitry.

• NME1212SC Isolated DC–DC Converter

- Generates isolated ±5 V supplies.
- Provides isolated power for the analog signal conditioning circuit, improving measurement accuracy and reducing electrical noise.

6. DB9 Communication Interface

The DB9 connector provides serial communication between the ultrasonic anemometer and an external computer or monitoring system.

• Pin Connections

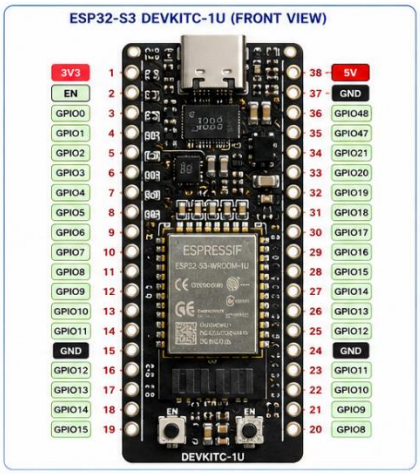
- **TXD:** Connected to ESP32 UART Transmit.
- **RXD:** Connected to ESP32 UART Receive.
- **GND:** Connected to system ground.

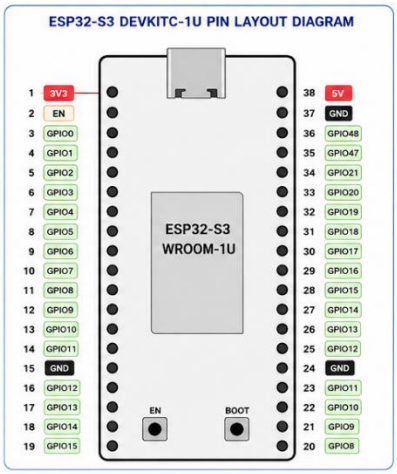
The DB9 interface allows measured wind speed and wind direction data to be transmitted to external software for monitoring and analysis.

ESP32-S3 PIN CONFIGURATION

- The ESP32-S3 DevKitC-1U is a development board based on the ESP32-S3-WROOM-1U module, designed for embedded and IoT applications.

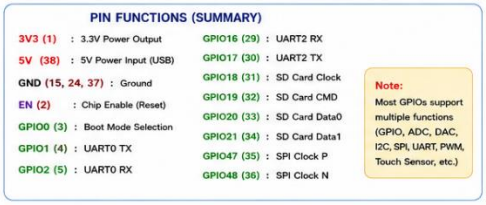
- The board operates at a 3.3 V logic level and is powered through the USB Type-C connector or the 5 V input pin.

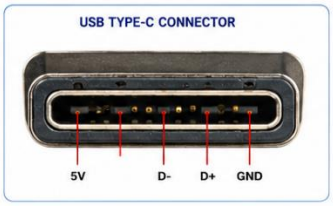


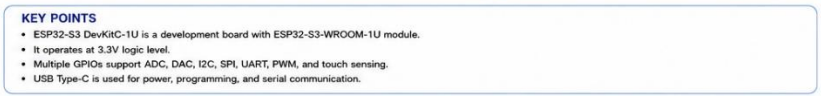


ESP32-S3 PIN DESCRIPTION

Pin No.	Signal	Description	Direction
1	3V3	3.3V Power Output	–
2	EN	Chip Enable (Active High)	Input
3	GPIO0	Boot Strapping Pin /	Input/Output
4	GPIO1	U0TXD (UART0 Transmit)	Output
5	GPIO2	U0RXD (UART0 Receive)	Input
6	GPIO3	GPIO / ADC1_CH2	Input/Output
7	GPIO4	GPIO / ADC1_CH3	Input/Output
8	GPIO5	GPIO / ADC1_CH4	Input/Output
9	GPIO6	GPIO / ADC1_CH5	Input/Output
10	GPIO7	GPIO / ADC1_CH6	Input/Output
11	GPIO8	GPIO / ADC1_CH7	Input/Output
12	GPIO9	GPIO / ADC1_CH8	Input/Output
13	GPIO10	GPIO / ADC2_CH0	Input/Output
14	GPIO11	GPIO / ADC2_CH1	Input/Output
15	GND	Ground	–
16	GPIO12	GPIO / ADC2_CH2	Input/Output
17	GPIO13	GPIO / ADC2_CH3	Input/Output
18	GPIO14	GPIO / ADC2_CH4	Input/Output
19	GPIO15	GPIO / ADC2_CH5 / MTDO	Input/Output
20	GPIO8	GPIO / ADC1_CH7 (Re-used)	Input/Output
21	GPIO9	GPIO / ADC1_CH8 (Re-used)	Input/Output
22	GPIO10	GPIO / ADC2_CH0 (Re-used)	Input/Output
23	GPIO11	GPIO / ADC2_CH1 (Re-used)	Input/Output
24	GND	Ground	–
25	GPIO12	GPIO / ADC2_CH2 (Re-used)	Input/Output
26	GPIO13	GPIO / ADC2_CH3 (Re-used)	Input/Output
27	GPIO14	GPIO / ADC2_CH4 (Re-used)	Input/Output
28	GPIO15	GPIO / ADC2_CH5 / MTDO (Re-used)	Input/Output
29	GPIO16	GPIO / U2RXD (UART2 Receive)	Input/Output
30	GPIO17	GPIO / U2TXD (UART2 Transmit)	Output
31	GPIO18	GPIO / SD_CLK	Input/Output
32	GPIO19	GPIO / SD_CMD	Input/Output
33	GPIO20	GPIO / SD_DATA0	Input/Output
34	GPIO21	GPIO / SD_DATA1	Input/Output
35	GPIO47	GPIO / SPICLK_P / CLK_OUT2	Input/Output
36	GPIO48	GPIO / SPICLK_N / CLK_OUT3	Input/Output
37	GND	Ground	–
38	5V	5V Power Input	–





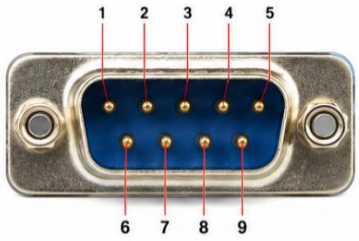


DB9 SERIAL COMMUNICATION INTERFACE

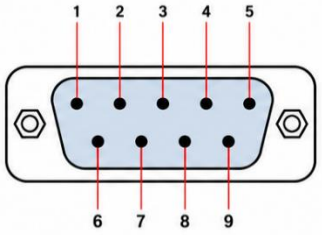
- The DB9 connector provides a standard RS-232 serial communication interface between the ESP32-S3 system and external devices.
- The DB9 connector enables communication between the ESP32-S3-based ultrasonic anemometer and external systems such as a PC, PLC, HMI, data logger, or industrial controller.

RS-232 (DB9) PIN CONFIGURATION

DB9 CONNECTOR (FRONT VIEW – MALE)



DB9 CONNECTOR (PIN LAYOUT DIAGRAM)



RS-232 (DB9) PIN DESCRIPTION

Pin No.	Signal	Full Form	Direction
1	DCD	Data Carrier Detect	Input
2	RXD	Receive Data	Input
3	TXD	Transmit Data	Output
4	DTR	Data Terminal Ready	Output
5	GND	Signal Ground	–
6	DSR	Data Set Ready	Input
7	RTS	Request To Send	Output
8	CTS	Clear To Send	Input
9	RI	Ring Indicator	Input

RS-232 PIN FUNCTIONS (SUMMARY)

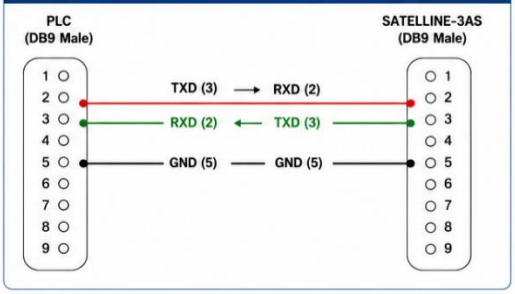
RXD (2) : Receives data from the other device.
TXD (3) : Transmits data to the other device.
GND (5) : Signal Ground (common reference).
DTR (4) : Indicates that the DTE is ready.
RTS (7) : Request to send data.
CTS (8) : Clear to send data.
DCD (1) : Data carrier detect.
DSR (6) : Data set ready.
RI (9) : Ring indicator.

Note:
Most common communication uses only Pin 2 (RXD), Pin 3 (TXD) and Pin 5 (GND).

RS-232 CONNECTOR (MALE)

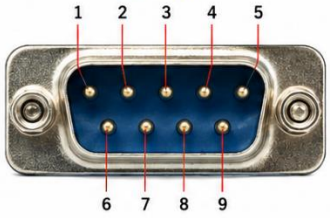


RS-232 CONNECTION (PLC ↔ SATELLINE-3AS)



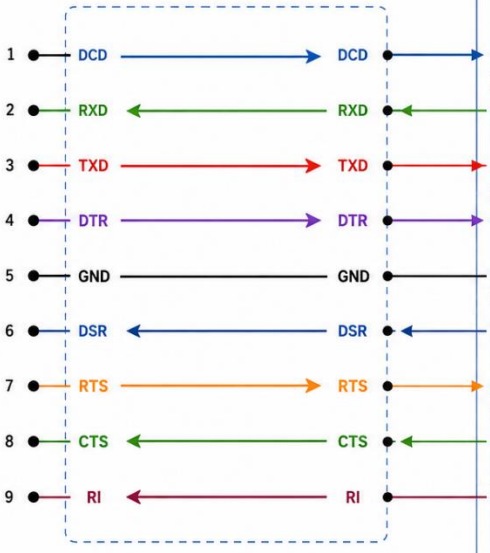
DB9 TO EXTERNAL DEVICES CONFIGURATION

DB9 MALE CONNECTOR (RS-232)



PIN NO.	SIGNAL	DESCRIPTION	DIRECTION
1	DCD	Data Carrier Detect	Input
2	RXD	Receive Data	Input
3	TXD	Transmit Data	Output
4	DTR	Data Terminal Ready	Output
5	GND	Signal Ground	—
6	DSR	Data Set Ready	Input
7	RTS	Request To Send	Output
8	CTS	Clear To Send	Input
9	RI	Ring Indicator	Input

DB9 TO EXTERNAL DEVICES CONNECTIONS



EXTERNAL DEVICES

PC / COMPUTER
- Serial Communication
- Data Exchange
- Configuration

PLC / CONTROLLER
- Industrial Control
- Monitoring
- Data Logging

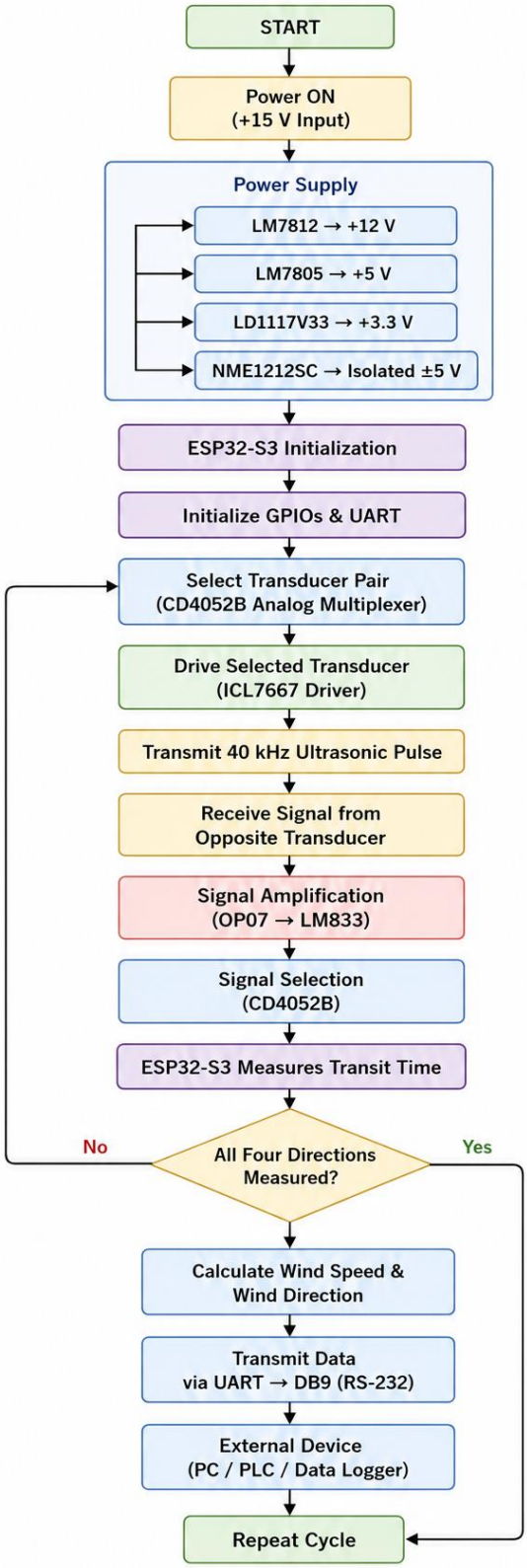
MODEM
- Remote Communication
- Data Transmission

DATA LOGGER
- Data Acquisition
- Environmental Monitoring
- Storage

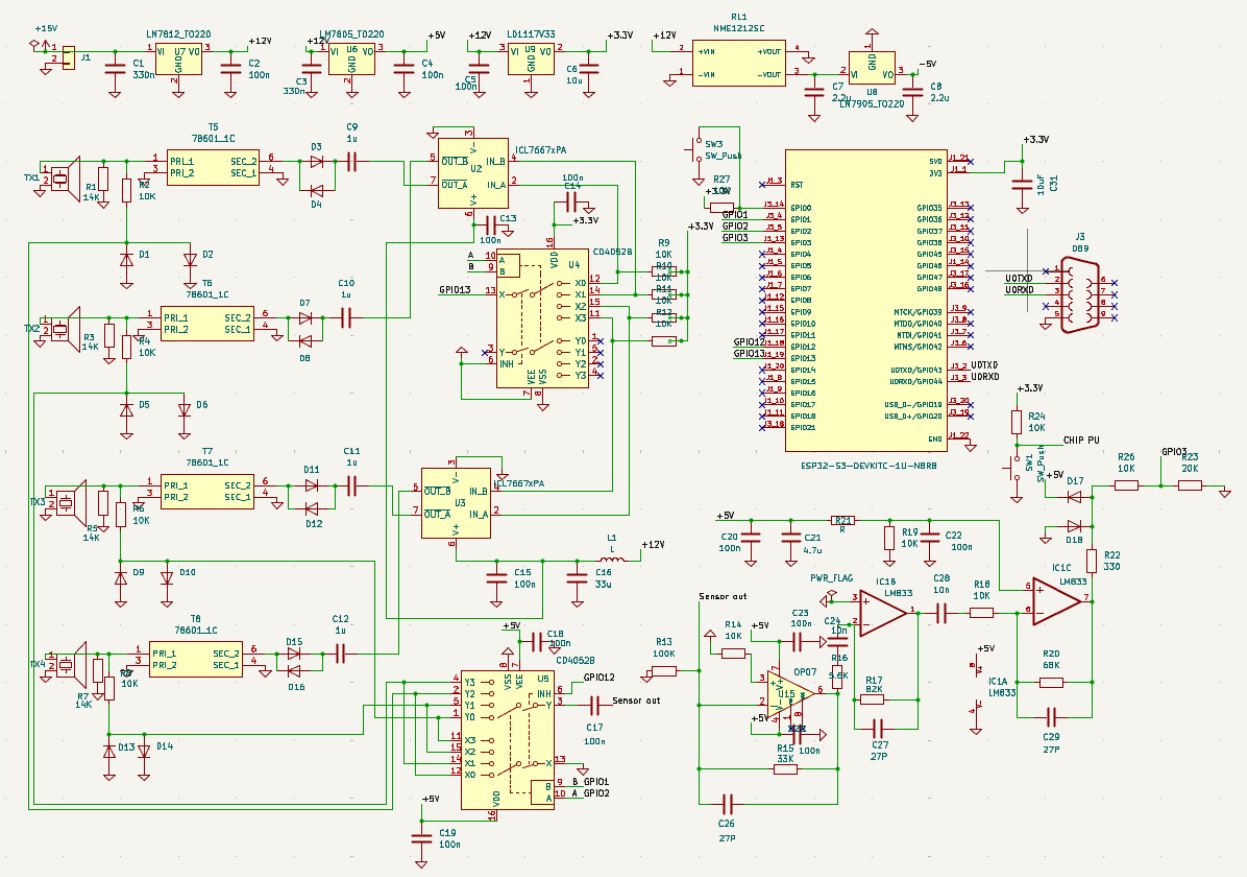
SENSOR INTERFACE
- Sensor Data Transfer
- Real-time Monitoring

OTHER RS-232 DEVICES
- HMI / Display
- Serial Devices
- Custom Systems

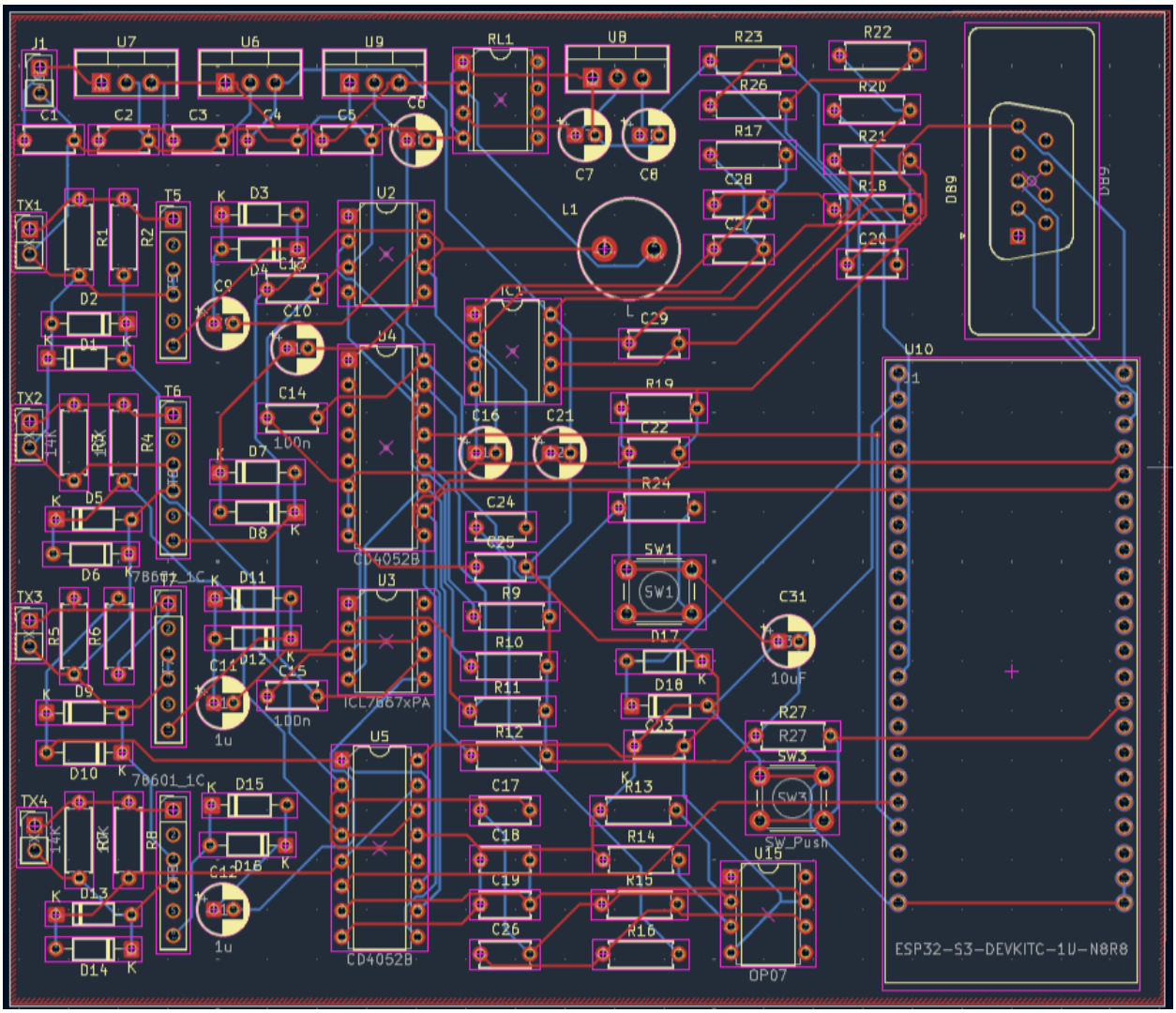
FLOWCHART


START
Power ON
(+15 V Input)
Power Supply
LM7812 → +12 V
LM7805 → +5 V
LD1117V33 → +3.3 V
NME1212SC → Isolated ±5 V
ESP32-S3 Initialization
Initialize GPIOs & UART
Select Transducer Pair
(CD4052B Analog Multiplexer)
Drive Selected Transducer
(ICL7667 Driver)
Transmit 40 kHz Ultrasonic Pulse
Receive Signal from
Opposite Transducer
Signal Amplification
(OP07 → LM833)
Signal Selection
(CD4052B)
ESP32-S3 Measures Transit Time
No
All Four Directions
Measured?
Yes
Calculate Wind Speed &
Wind Direction
Transmit Data
via UART → DB9 (RS-232)
External Device
(PC / PLC / Data Logger)
Repeat Cycle

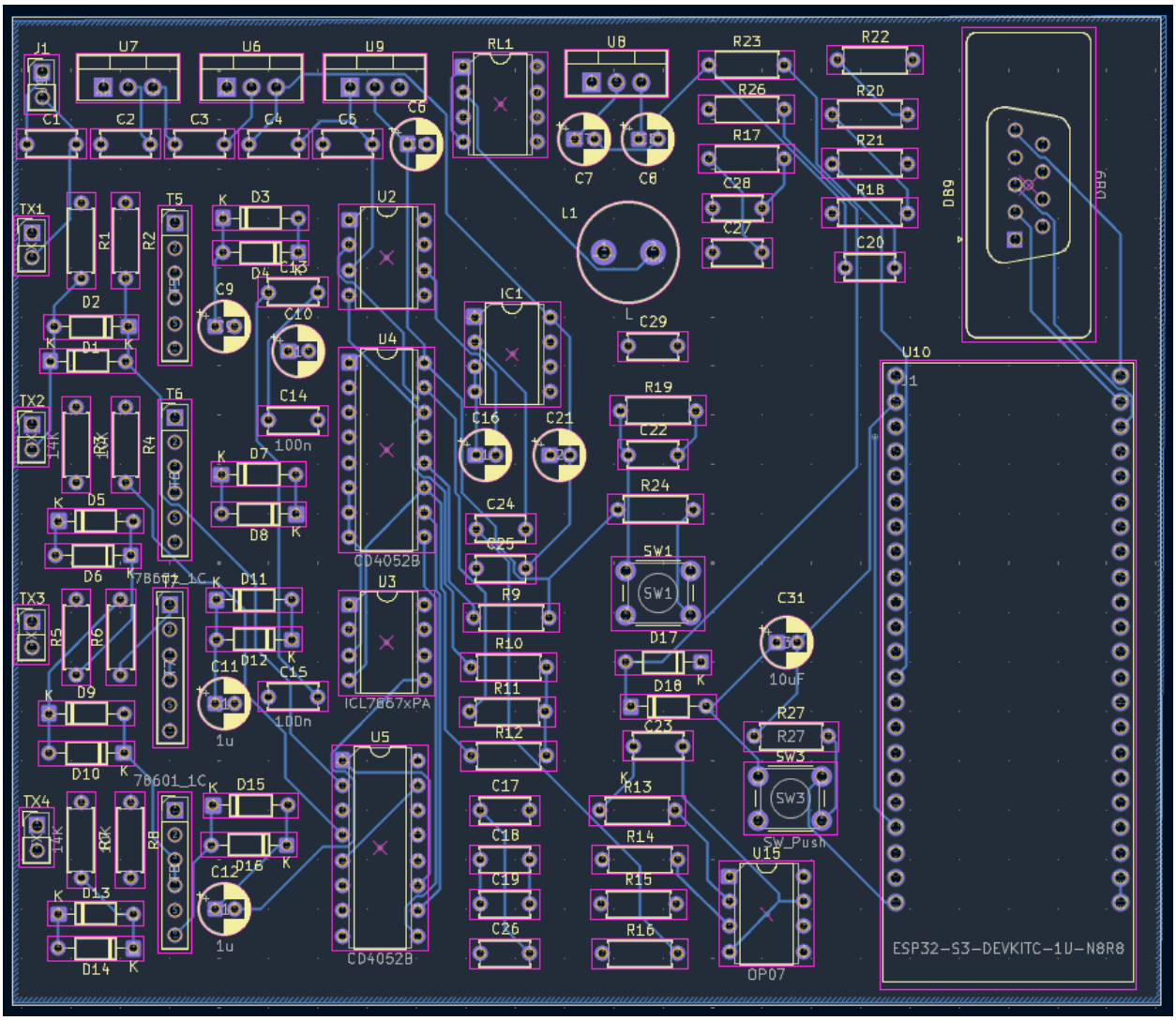
SCHEMATIC LAYOUT



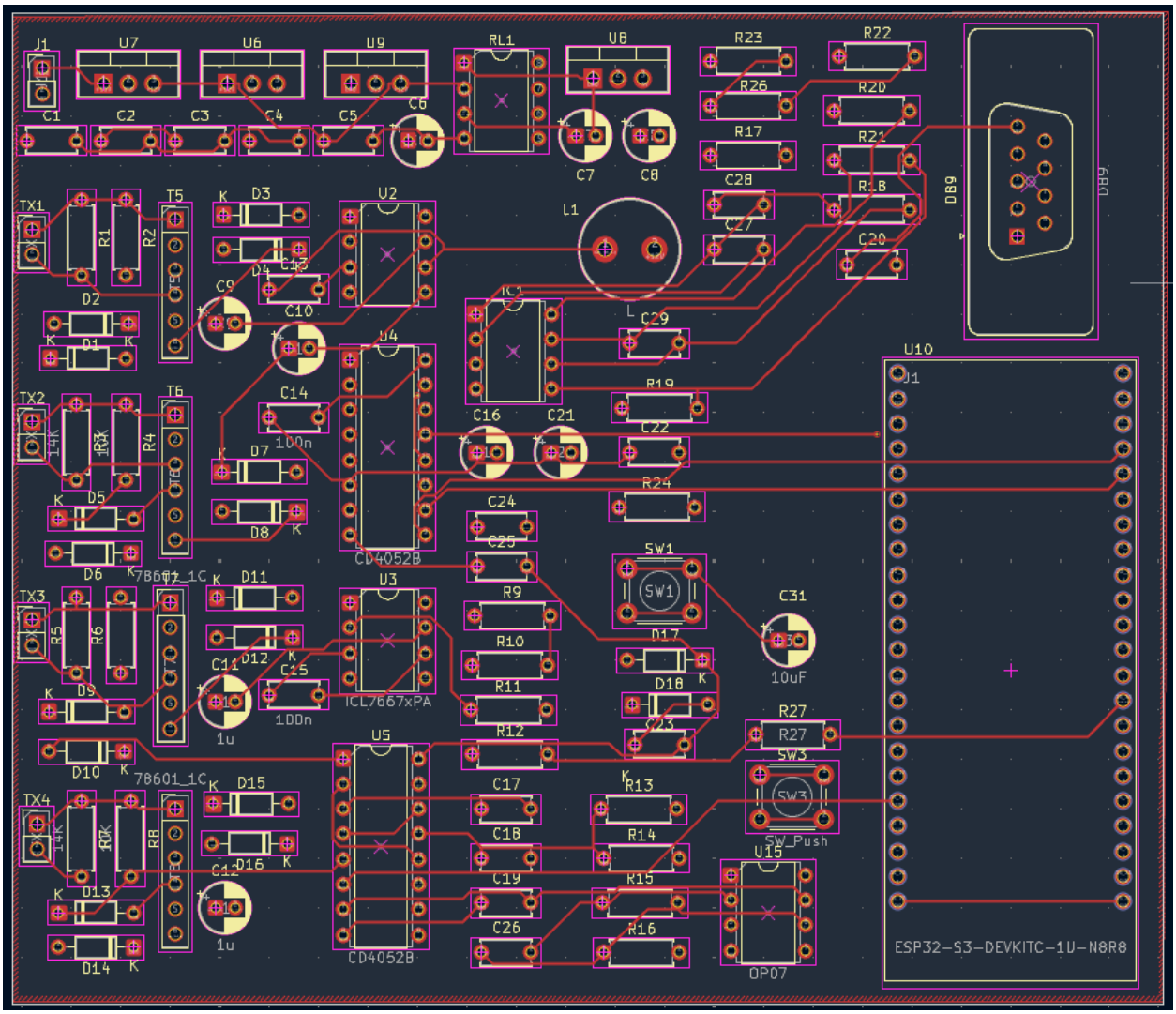
PCB LAYOUT


J1
U7
U6
U9
RL1
U8
R23
R22
R26
R20
R17
R21
R18
C28
C27
C20
DB9
C1
C2
C3
C4
C5
C6
C7
C8
L1
TX1
T5
D3
U2
D4
C13
C9
C10
D2
D1
IC1
C29
U10
TX2
T6
C14
100n
C16
C21
R19
C22
D7
R24
D5
D8
C24
C25
SW1
D6
78601_1C
U3
C31
TX3
D11
R5
R6
R9
10uF
D12
C11
C15
R10
D17
D9
1u
100n
ICL7667xPA
R11
D18
R27
D10
U5
R12
C23
R27
SW3
TX4
D15
C17
R13
C18
R14
R8
C12
D16
SW_Push
U15
D13
C19
R15
1u
C26
R16
D14
CD4052B
OP07
ESP32-S3-DEVKITC-1U-N8R8

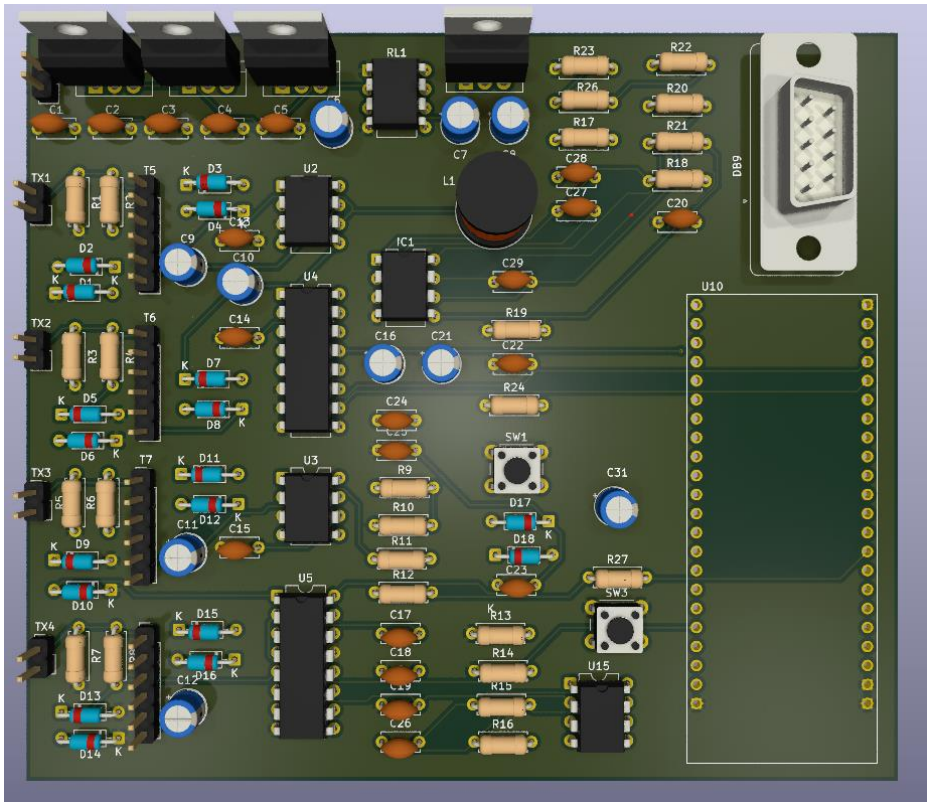
TOP LAYER PCB ROUTING

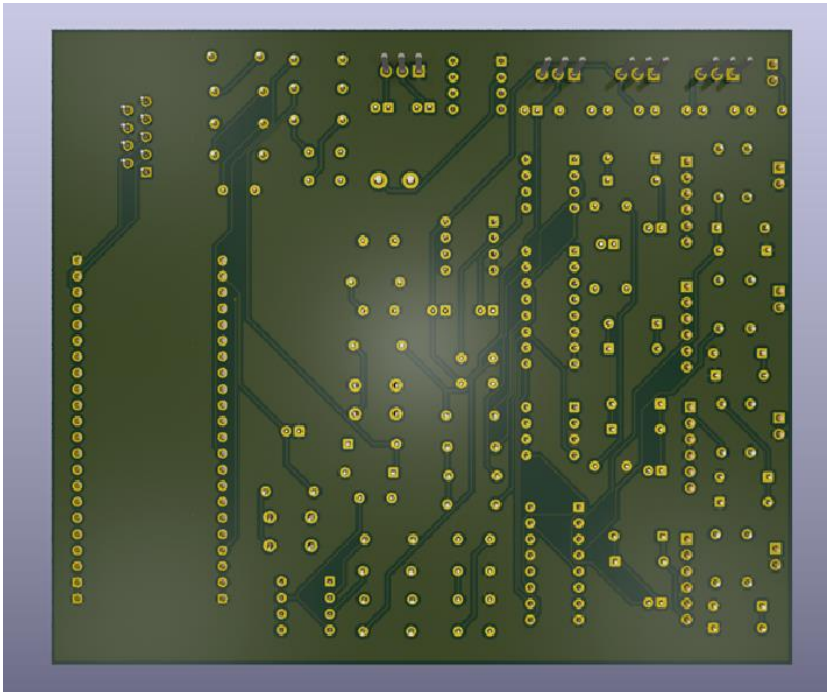


BOTTOM LAYER PCB ROUTING

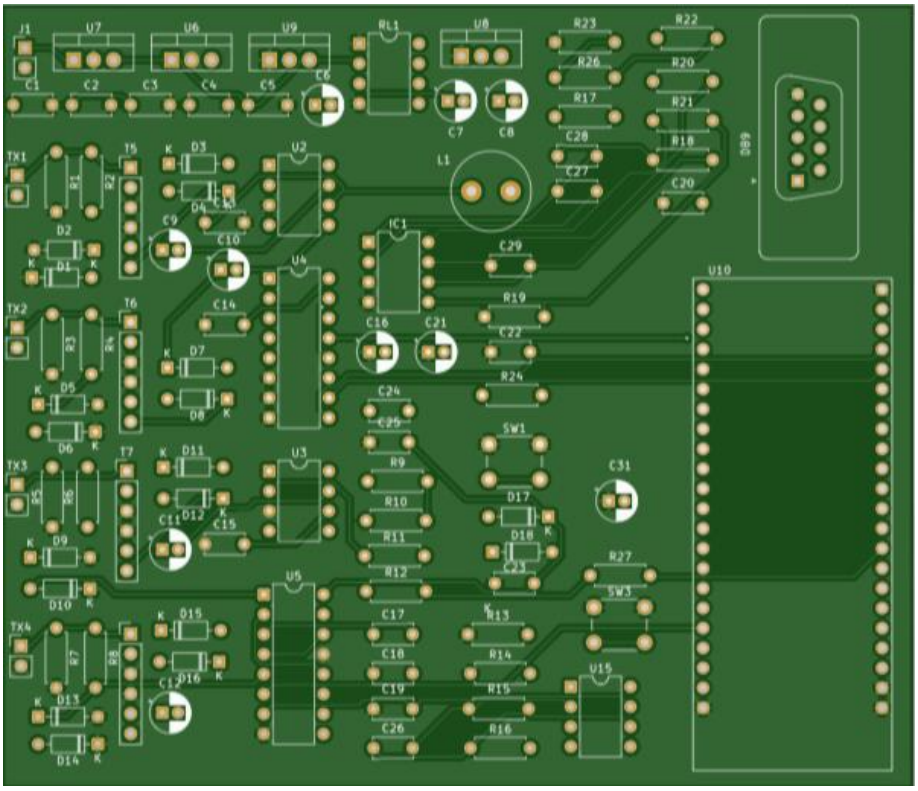


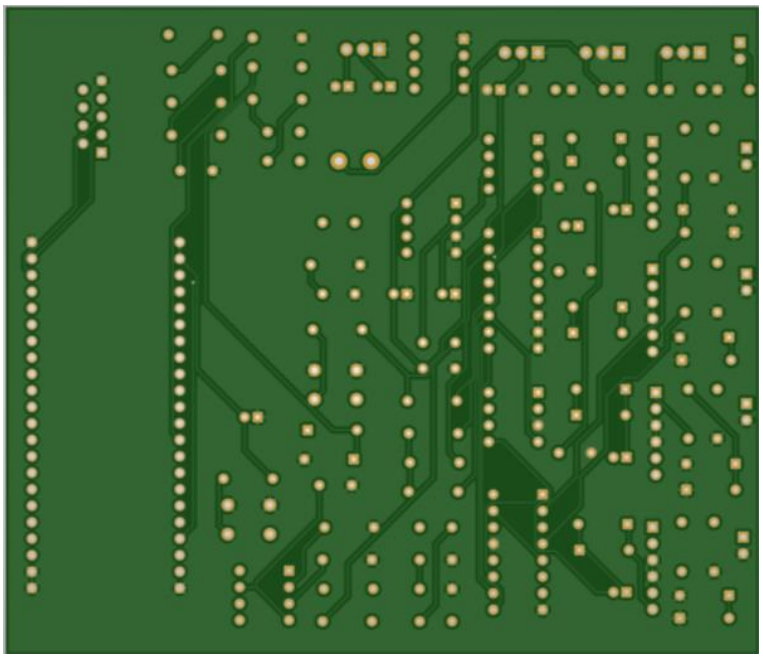
3D PCB LAYOUT





GERBER LAYOUT





PROGRAM

```
#include <Arduino.h>
#define SOUND_SPEED_20C     343.2f
#define TRANSDUCER_DISTANCE 0.150f       // 150 mm
#define PULSE_FREQUENCY     40000
#define PULSE_COUNT         16
const int TX_MUX_A = 13;
const int RX_MUX_A = 12;
const int MUX_S0 = 1;
const int MUX_S1 = 2;
const int DRIVER_ENABLE = 5;
const int RX_SIGNAL = 4;
const int PWM_CHANNEL = 0;
const int PWM_RESOLUTION = 8;
const int PWM_FREQUENCY = 40000;

enum Direction
{
   NORTH = 0,
   EAST,
   SOUTH,
   WEST
};
struct Measurement
{
   float transitTime;
   float velocity;
};
Measurement NS;
Measurement SN;
Measurement EW;
Measurement WE;

float windSpeed = 0.0;
float windDirection = 0.0;
unsigned long lastMeasurement = 0;
const unsigned long SAMPLE_PERIOD = 100;
void initializeHardware();
void generateBurst();
```

```
void selectTransmit(Direction dir);
void selectReceive(Direction dir);
float measureTransit(Direction tx, Direction rx);
void calculateWind();
void printTelemetry();
void setup()
{
  Serial.begin(115200);
  initializeHardware();
  Serial.println("-----------------------------------");
  Serial.println("ULTRASONIC ANEMOMETER");
  Serial.println("ESP32-S3 INITIALIZED");
  Serial.println("-----------------------------------");
}

void loop()
{
  if (millis() - lastMeasurement >= SAMPLE_PERIOD)
  {
    lastMeasurement = millis();
    // North -> South
    NS.transitTime = measureTransit(NORTH, SOUTH);
    // South -> North
    SN.transitTime = measureTransit(SOUTH, NORTH);
    // East -> West
    EW.transitTime = measureTransit(EAST, WEST);
    // West -> East
    WE.transitTime = measureTransit(WEST, EAST);
    calculateWind();
    printTelemetry();
  }
}

void initializeHardware()
{
  pinMode(TX_MUX_A, OUTPUT);
  pinMode(RX_MUX_A, OUTPUT);
  pinMode(MUX_S0, OUTPUT);
  pinMode(MUX_S1, OUTPUT);
  pinMode(DRIVER_ENABLE, OUTPUT);
```

```
    pinMode(RX_SIGNAL, INPUT);
    digitalWrite(DRIVER_ENABLE, HIGH);
    ledcSetup(
        PWM_CHANNEL,
        PWM_FREQUENCY,
        PWM_RESOLUTION);
    ledcAttachPin(
        DRIVER_ENABLE,
        PWM_CHANNEL);
    ledcWrite(
        PWM_CHANNEL,
        128);
}
void selectTransmit(Direction dir)
{
    digitalWrite(TX_MUX_A, HIGH);
    switch(dir)
    {
        case NORTH:
            digitalWrite(MUX_S0,LOW);
            digitalWrite(MUX_S1,LOW);
            break;
        case EAST:
            digitalWrite(MUX_S0,HIGH);
            digitalWrite(MUX_S1,LOW);
            break;
        case SOUTH:
            digitalWrite(MUX_S0,LOW);
            digitalWrite(MUX_S1,HIGH);
            break;

        case WEST:
            digitalWrite(MUX_S0,HIGH);
            digitalWrite(MUX_S1,HIGH);
            break;
    }
}

void selectReceive(Direction dir)
{
```

```
  digitalWrite(RX_MUX_A, HIGH);

  switch(dir)
  {
    case NORTH:
      digitalWrite(MUX_S0,LOW);
      digitalWrite(MUX_S1,LOW);
      break;

    case EAST:
      digitalWrite(MUX_S0,HIGH);
      digitalWrite(MUX_S1,LOW);
      break;
    case SOUTH:
      digitalWrite(MUX_S0,LOW);
      digitalWrite(MUX_S1,HIGH);
      break;
    case WEST:
      digitalWrite(MUX_S0,HIGH);
      digitalWrite(MUX_S1,HIGH);
      break;
  }
}
void generateBurst()
{
  ledcWrite(PWM_CHANNEL,128);
  delayMicroseconds(400);
  ledcWrite(PWM_CHANNEL,0);
}

float measureTransit(Direction tx, Direction rx)
{
  selectTransmit(tx);
  selectReceive(rx);
  delayMicroseconds(20);
  generateBurst();
  unsigned long startTime = micros();
  while(digitalRead(RX_SIGNAL)==LOW)
  {
    if(micros()-startTime>5000)
```

```
        {
            return -1;
        }
    }
    unsigned long stopTime = micros();
    return (float)(stopTime-startTime);
}
void calculateWind()
{
    float tNS = NS.transitTime*1e-6;
    float tSN = SN.transitTime*1e-6;
    float tEW = EW.transitTime*1e-6;
    float tWE = WE.transitTime*1e-6;
    if(tNS<=0 || tSN<=0 || tEW<=0 || tWE<=0)
        return;
    float vx = (TRANSDUCER_DISTANCE/2.0f)*
            ((1.0f/tEW)-(1.0f/tWE));

    float vy = (TRANSDUCER_DISTANCE/2.0f)*
            ((1.0f/tNS)-(1.0f/tSN));

    windSpeed = sqrt(vx*vx + vy*vy);
    windDirection = atan2(vx,vy);
    windDirection *= 180.0/PI;
    if(windDirection<0)
        windDirection+=360.0;
}
void printTelemetry()
{
    Serial.print("NS : ");
    Serial.print(NS.transitTime);
    Serial.print(" us   ");
    Serial.print("SN : ");
    Serial.print(SN.transitTime);
    Serial.print(" us   ");
    Serial.print("EW : ");
    Serial.print(EW.transitTime);
    Serial.print(" us   ");
    Serial.print("WE : ");
    Serial.print(WE.transitTime);
```

```
    Serial.print(" us   ");
    Serial.print("Speed : ");
    Serial.print(windSpeed);
    Serial.print(" m/s   ");
    Serial.print("Direction : ");
    Serial.print(windDirection);
    Serial.println(" deg");
}
#ifndef SIGNAL_PROCESSING_H
#define SIGNAL_PROCESSING_H
#include <Arduino.h>
#define ADC_BUFFER_SIZE
#define FILTER_SIZE
struct SignalResult
{
    bool detected;
    uint32_t arrivalTime;
    float amplitude;
    float noise;
};
class SignalProcessor
{
public:
    SignalProcessor();
    void begin();
    void clearBuffer();
    void acquire();
    void filter();
    bool detect();
    uint32_t getArrivalTime();
    float getPeakAmplitude();
    float getNoiseLevel();
    SignalResult process();
private:
    uint16_t adcBuffer[ADC_BUFFER_SIZE];
    float filteredBuffer[ADC_BUFFER_SIZE];
    uint32_t arrivalTime;
    float peak;
    float noise;
};
```

```
extern SignalProcessor DSP;
#endif
#include "signal_processing.h"
const int RX_ADC = 4;
SignalProcessor DSP;
SignalProcessor::SignalProcessor()
{
  arrivalTime = 0;
  peak = 0;
  noise = 0;
}
void SignalProcessor::begin()
{
  analogReadResolution(12);
  pinMode(RX_ADC, INPUT);
}
void SignalProcessor::clearBuffer()
{
  for(int i=0;i<ADC_BUFFER_SIZE;i++)
  {
    adcBuffer[i]=0;
    filteredBuffer[i]=0;
  }
  arrivalTime=0;
  peak=0;
  noise=0;
}
void SignalProcessor::acquire()
{
  for(int i=0;i<ADC_BUFFER_SIZE;i++)
  {
    adcBuffer[i]=analogRead(RX_ADC);
  }
}
void SignalProcessor::filter()
{
  for(int i=FILTER_SIZE;i<ADC_BUFFER_SIZE;i++)
  {
    float sum=0;
```

```
        for(int j=0;j<FILTER_SIZE;j++)
            sum+=adcBuffer[i-j];

        filteredBuffer[i]=sum/FILTER_SIZE;
    }
}

bool SignalProcessor::detect()
{
    peak=0;

    for(int i=0;i<ADC_BUFFER_SIZE;i++)
    {
        if(filteredBuffer[i]>peak)
            peak=filteredBuffer[i];
    }

    noise=0;
    for(int i=0;i<50;i++)
        noise+=filteredBuffer[i];
    noise/=50;
    float threshold=noise+((peak-noise)*0.35);
    for(int i=0;i<ADC_BUFFER_SIZE;i++)
    {
        if(filteredBuffer[i]>threshold)
        {
            arrivalTime=i;
            return true;
        }
    }
    return false;
}
uint32_t SignalProcessor::getArrivalTime()
{
    return arrivalTime;
}
float SignalProcessor::getPeakAmplitude()
{
    return peak;
```

```
}
float SignalProcessor::getNoiseLevel()
{
   return noise;
}
SignalResult SignalProcessor::process()
{
   SignalResult result;
   clearBuffer();
   acquire();
   filter();
   result.detected=detect();
   result.arrivalTime=arrivalTime;
   result.amplitude=peak;
   result.noise=noise;
   return result;
}
#ifndef TRANSDUCER_H
#define TRANSDUCER_H
#include <Arduino.h>
enum Transducer
{
   NORTH=0,
   EAST,
   SOUTH,
   WEST
};
class Ultrasonic
{
public:

   void begin();
   void transmit(Transducer tx);
   float receive(Transducer rx);
   float measure(Transducer tx,Transducer rx);
private:
   void selectTX(Transducer tx);
   void selectRX(Transducer rx);
   void burst();
```

```
};
extern Ultrasonic UltrasonicSensor;
#endif
#include "transducer.h"
#include "signal_processing.h"
Ultrasonic UltrasonicSensor;
extern const int TX_MUX_A;
extern const int RX_MUX_A;
extern const int MUX_S0;
extern const int MUX_S1;
extern const int PWM_CHANNEL;
void Ultrasonic::begin()
{
   DSP.begin();
}
void Ultrasonic::selectTX(Transducer tx)
{
   digitalWrite(TX_MUX_A,HIGH);
   digitalWrite(MUX_S0,tx&1);
   digitalWrite(MUX_S1,(tx>>1)&1);
}
void Ultrasonic::selectRX(Transducer rx)
{
   digitalWrite(RX_MUX_A,HIGH);
   digitalWrite(MUX_S0,rx&1);
   digitalWrite(MUX_S1,(rx>>1)&1);
}

void Ultrasonic::burst()
{
   ledcWrite(PWM_CHANNEL,128);
   delayMicroseconds(400);
   ledcWrite(PWM_CHANNEL,0);
}
void Ultrasonic::transmit(Transducer tx)
{
   selectTX(tx);
   burst();
}
```

```
float Ultrasonic::receive(Transducer rx)
{
   selectRX(rx);
   SignalResult r=DSP.process();
   if(!r.detected)
      return -1;
   return r.arrivalTime;
}
float Ultrasonic::measure(Transducer tx,Transducer rx)
{
   transmit(tx);
   delayMicroseconds(10);
   return receive(rx);
}
```